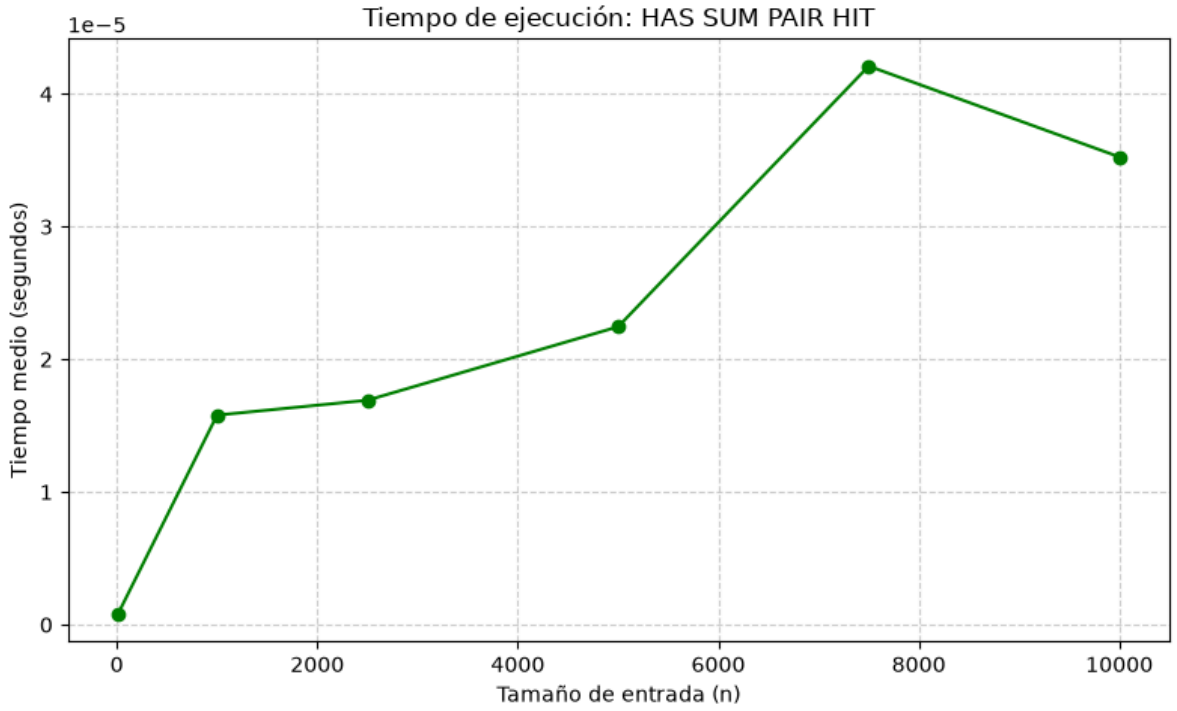
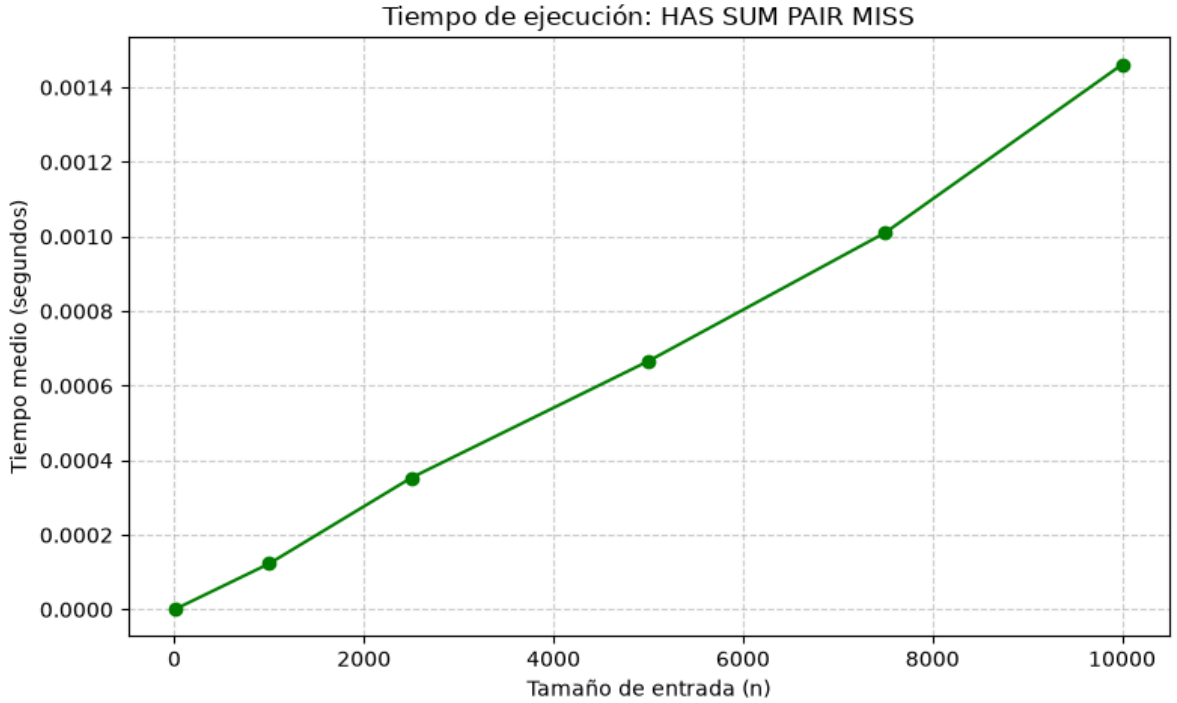
Gráfica Has_sum_pair_hit:



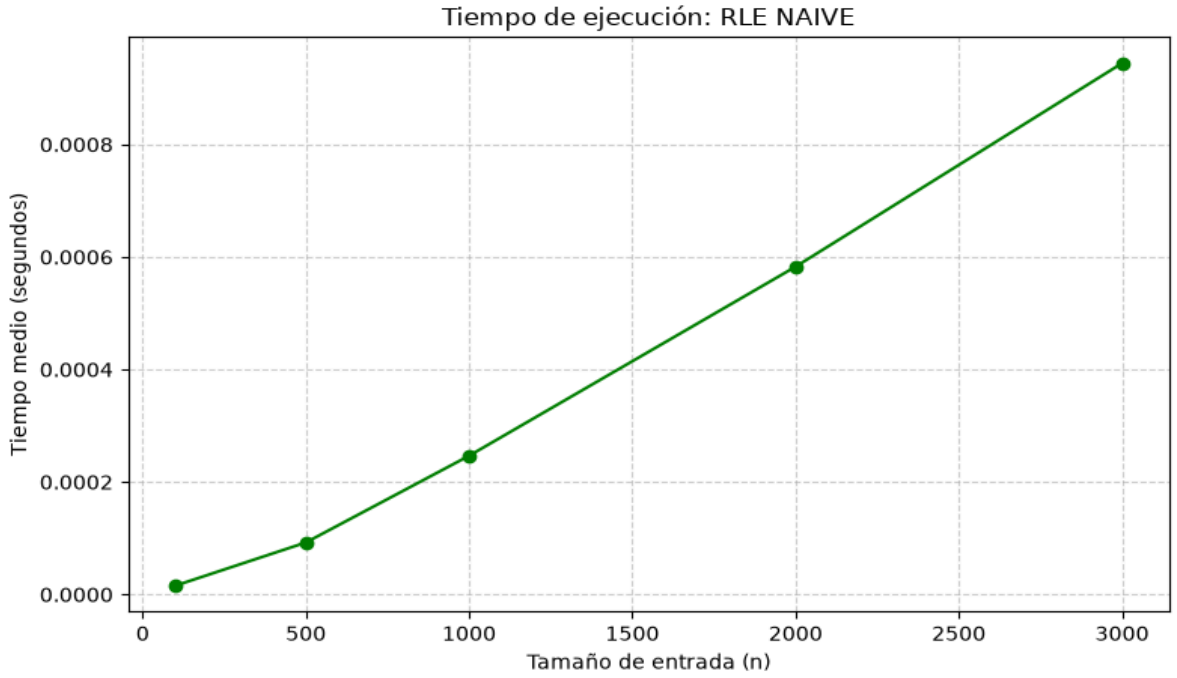
Gráfica Has_sum_pait_miss:



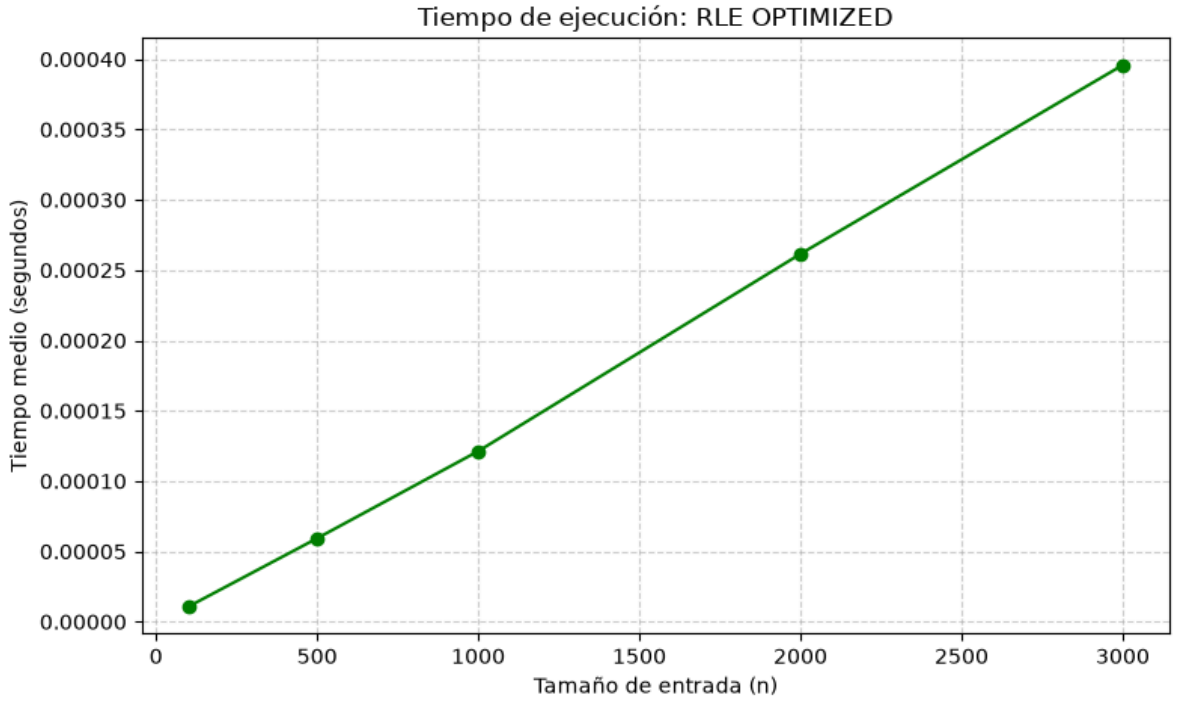
Como podemos observar en las gráficas, la primera tiene un tiempo de ejecución menor y que no aumenta tanto cuando aumentamos el tamaño de entrada, esto se debe a que el tiempo de ejecución varía ya que a veces encuentra la pareja que suma target y termina de buscar.

En cambio en la segunda siempre recorre todas las posibilidades y nunca encuentra la pareja lo que mantiene el tiempo de ejecución mayor y con crecimiento más constante siendo proporcional a n.

Gráfica RLE Naive:



Gráfica RLE Optimized:



En estas gráficas podemos ver la diferencia de tiempo debido a que la concatenación se hace de formas distintas, al usar el operador + (lista = lista + [...]) lo que crea una nueva lista haciendo que eleve la complejidad a $O(n^2)$. Al usar .append en la otra función, reduce el tiempo de ejecución considerablemente y la complejidad a O(n)